Universidad Nacional de Lomas de Zamora

INFORME DE TRABAJO PRÁCTICO

Creación de API REST con Flask, SQLAlchemy, Marshmallow

Materia: Taller Integrador
Profesor: Rodrigo Lema

Alumna:

- Rebeca Anahi Luna Colque

15 de abril de 2026

INFORME DE TRABAJO PRÁCTICO

1.- Investigación de las tecnologías:

Para el desarrollo del proyecto, se realizó una fase de investigación y capacitación técnica mediante los siguientes recursos:

- **Flask:** Se utilizaron los materiales audiovisuales proporcionados en clase, practicando la configuración de entornos virtuales (`.venv`)
- **SQLite:** Se complementa el aprendizaje con tutoriales externos para la gestión de bases de datos relacionales ligeras.
  - *Recurso de referencia:* Video Tutorial SQLite
  - *Repositorio de práctica:* GitHub - SQLITE_PRACTICA
- **SQLAlchemy y Marshmallow:** Se investigó la implementación del ORM para la manipulación de datos y Marshmallow para la serialización de objetos.
  - *Recurso de referencia:* Video Tutorial SQLAlchemy/Marshmallow
  - *Repositorio de práctica:* GitHub - Practica SQLAlchemy/Marshmallow
- **Dockerfile:** Se abordaron conceptos básicos de contenedores y archivos `Dockerfile`. Aunque se logró una base conceptual, el manejo avanzado de esta tecnología sigue en proceso de fortalecimiento.
  - *Recurso de referencia:* Video Tutorial Docker

2.-Implementación de Inteligencia Artificial:

2.1 Consultas técnicas:

Durante la práctica, realicé consultas para entender los errores que surgían y, al mismo tiempo, solicitar explicaciones sobre su causa. En uno de los casos, el inconveniente se debía a que el contenido de un video tutorial estaba desactualizado. Por otro lado, también realicé consultas para profundizar en la comprensión de distintos conceptos técnicos.

---

versión antigua (video)

```
class ProductSchema(ma.Schema):
    class Meta:
        fields = ('id','name','description','price','qty')
```

versión actualizada

```python
class ProductSchema(ma.SQLAlchemyAutoSchema):
    class Meta:
        model = Product
        load_instance = True
```

Mi comentario según mi comprensión

```
class ProductSchema(ma.SQLAlchemyAutoSchema):
    class Meta:
        model = Product #Campos q quiero mostrar
        load_instance = True # si quiero usar objetos
```

Corrección

```
class ProductSchema(ma.SQLAlchemyAutoSchema):
    class Meta:
        model = Product  # Usa automáticamente los campos del modelo Product
        load_instance = True  # Convierte JSON en objetos Product
```

2.2 Estructura de carpetas:

En base al ejemplo del PDF sobre la estructura de proyectos, armé una estructura inicial. Luego le consulté a una IA qué le parecía, y me sugirió una alternativa. Además, le compartí mi código de práctica, que estaba todo en un solo archivo (app.py), por lo que, a partir de ese código y de mi estructura inicial, me propuso una nueva organización de carpetas.

Estructura inicial

```
/ ├── README.md
 ├── /backend # Backend Flask (auto-contenido)
 │   ├── venv/ # Entorno virtual (no se sube al repo)
 │   ├── requirements.txt # Dependencias del backend
 │   ├── app.py # Entrada principal del backend
 │   ├── /routes # Rutas o Blueprints
 │   ├── /models # Modelos de datos (SQLAlchemy)
 │   └── /services # Lógica de negocio
 ├── /docs # Diagramas, flujos, convenciones propios del proyecto
 └── /deploy # Dockerfiles, CI/CD, scripts de despliegue
```

Estructura de la IA

```
backend/
│
├── app.py
├── extensions.py
│
├── models/
│   └── product_model.py
│
├── schemas/
│   └── product_schema.py
│
├── services/
│   └── product_service.py
│
```

```
├── controllers/
│   └── product_controller.py
│
├── routes/
│   └── product_routes.py
```

Luego ahí construí mi estructura de carpetas y le agregue las demás carpetas y archivos que necesitaba para el TP.

2.3 Gestión de Importaciones:

Cuando le compartí mi código de práctica para el armado de la estructura de carpetas, no solo me indicó cómo organizar las rutas, sino que también me proporcionó ejemplos de cómo realizar las importaciones entre los distintos archivos para poder reutilizarlos.

Por ejemplo, el archivo product_service.py necesitaba utilizar el modelo Product, por lo que debía importarse desde models/product_model.py. Este mismo criterio se aplicaba al resto de los archivos según sus dependencias.

Otro ejemplo:

extensions.py

```python
from flask_sqlalchemy import SQLAlchemy
from flask_marshmallow import Marshmallow

db = SQLAlchemy()
ma = Marshmallow()
```

models/product_model.py

```python
from extensions import db

class Product(db.Model):
    id = db.Column(db.Integer,primary_key=True)
    name = db.Column(db.String(100),unique=True)
    description = db.Column(db.String(200))
    price = db.Column(db.Float)
    qty = db.Column(db.Integer)
```

2.4 Explicación de la relación entre tablas:

Al revisar la documentación de SQLAlchemy, no lograba comprender completamente cómo se establecían las relaciones entre tablas. Por este motivo, recurrí a una IA para que me explicara el funcionamiento de dichas relaciones y así poder aplicarlas correctamente en el proyecto.

2.5 Guía para el despliegue con Docker:

En el caso de Docker, contaba con un modelo base obtenido de un video tutorial, el cual utilicé como referencia para crear el **`Dockerfile`** del trabajo práctico. Posteriormente, solicité a la IA que verificará si la configuración era correcta, tanto para el **`Dockerfile`** como para **`docker-compose`**. A partir de las correcciones sugeridas, logré realizar el despliegue exitosamente.

2.6 Corrección de redacción:

Tanto para la parte técnica como para la redacción del informe y el archivo README, utilicé ChatGPT como herramienta de apoyo para mejorar la claridad y la presentación del contenido.

3.- Dificultades del proyecto:

Para comenzar el trabajo práctico, fue necesario capacitarme en las tecnologías utilizadas. Para ello, visualicé varios videos de entre 40 minutos y 1 hora de duración. Además del tiempo invertido en ver el contenido, también dediqué tiempo a seguir el código paso a paso y, en algunos casos, a comentar líneas para comprender mejor su funcionamiento.

Entre las principales dificultades, se encontró la comprensión de conceptos como *request*, *response* y *Blueprints*, entre otros. Asimismo, me resultó complicado dedicar el tiempo necesario para profundizar en estas tecnologías. Debido a los plazos de entrega, algunos temas, como Docker, no quedaron completamente claros.

No obstante, al contar con una base previa de código de práctica, pude utilizarla como guía para la organización de la estructura del proyecto y la correcta importación de módulos.

4.- Guia de uso:

Para utilizar el servicio, consulte el archivo [README](), donde se detalla el paso a paso necesario para su correcto uso.